目錄

程式碼截圖.....2

Verilog Code 設計流程簡述.....7

1. 共享加法器 (Resource Sharing).....7

2. 共享比較器.....7

3. 邊界處理.....7

4. FSM.....8

RTL sim 結果截圖(截到 CPU Time).....9

合成後 Report => timing_max 截圖.....10

GTL sim 結果截圖(截到 CPU Time).....12

心得(以組為單位，一篇就好).....13

程式碼截圖

```
module LBP (
    input              clk,
    input              reset,
    input       [7:0]  gray_data,

    output      [5:0]  gray_addr,
    output             gray_req,
    output reg  [5:0]  lbp_addr,
    output reg         lbp_write,
    output      [7:0]  lbp_data,
    output reg         finish
);


    localparam IDLE    = 2'd0,
               PROCESS = 2'd1,
               WRITE   = 2'd2,
               DONE    = 2'd3;

    reg [1:0] curr_state, next_state;
    reg [3:0] step;             // 0: 讀 gc, 1~8: 讀 g0~g7 並計算
    reg [5:0] center_addr;      // 目前中心像素的地址
    reg [7:0] gc;               // 僅儲存中心像素
    reg [7:0] lbp;              // 作為 Shift Register 使用
    reg [5:0] offset;           // 共享加法器的偏移量

```

```
// ==========================================
// 共享加法器與地址
// ==========================================
assign gray_req  = (curr_state == PROCESS && step <= 4'd8);
assign lbp_data = lbp;
assign gray_addr = center_addr + offset;

// 6-bit 補數
always @(*) begin
    case(step)
        4'd0: offset = 6'd0;   // 中心像素 (gc)
        4'd1: offset = 6'd55;  // -9 (g0)
        4'd2: offset = 6'd56;  // -8 (g1)
        4'd3: offset = 6'd57;  // -7 (g2)
        4'd4: offset = 6'd63;  // -1 (g3)
        4'd5: offset = 6'd1;   // +1 (g4)
        4'd6: offset = 6'd7;   // +7 (g5)
        4'd7: offset = 6'd8;   // +8 (g6)
        4'd8: offset = 6'd9;   // +9 (g7)
        default: offset = 6'd0;
    endcase
end
```

```
// ==========================================
// FSM
// ==========================================
always @(posedge clk or posedge reset) begin
    if(reset) curr_state <= IDLE;
    else      curr_state <= next_state;
end

always @(*) begin
    case(curr_state)
        IDLE:    next_state = PROCESS;
        PROCESS: next_state = (step == 4'd8) ? WRITE : PROCESS;
        WRITE:   begin
            // 是否處理完最後一個像素 (地址 54, 座標 6,6)
            if (center_addr == 6'd54) next_state = DONE;
            else                      next_state = PROCESS;
        end
        DONE:    next_state = DONE;
        default: next_state = PROCESS;
    endcase
end
```

```
// ==========================================
// Step Counter
// ==========================================
always @(posedge clk or posedge reset) begin
    if (reset)
        step <= 4'd0;
    else if (curr_state == PROCESS)
        step <= step + 1'b1;
    else
        step <= 4'd0;
end

// ==========================================
// center_addr
// ==========================================
always @(posedge clk or posedge reset) begin
    if (reset)
        center_addr <= 6'd9;
    else if (curr_state == WRITE) begin
        // 換行：x=6 時跳 3 格，否則跳 1 格
        if (center_addr[2:0] == 3'd6)
            center_addr <= center_addr + 6'd3;
        else
            center_addr <= center_addr + 6'd1;
    end
end
```

```
// ==========================================
// Center Pixel
// ==========================================
always @(posedge clk or posedge reset) begin
    if (reset)
        gc <= 8'd0;
    else if (curr_state == PROCESS && step == 4'd0)
        gc <= gray_data;
end

// ==========================================
// LBP Result
// ==========================================
always @(posedge clk or posedge reset) begin
    if (reset)
        lbp <= 8'd0;
    else if (curr_state == PROCESS)
        lbp <= { (gray_data >= gc), lbp[7:1] };
end
```

```
    // ==========================================
    // lbp_addr & lbp_write
    // ==========================================
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            lbp_addr  <= 6'd0;
            lbp_write <= 1'b0;
        end else if (curr_state == WRITE) begin
            lbp_addr  <= center_addr;
            lbp_write <= 1'b1;
        end else begin
            lbp_write <= 1'b0;
        end
    end


    // ==========================================
    // finish
    // ==========================================
    always @(posedge clk or posedge reset) begin
        if (reset)
            finish <= 1'b0;
        else if (curr_state == DONE)
            finish <= 1'b1;
        else
            finish <= 1'b0;
    end

endmodule
```

Verilog Code 設計流程簡述

我把之前的程式針對面積優化硬體。針對面積（Area）進行壓縮，並使用 compile_ultra 進一步壓面積

1. 共享加法器（Resource Sharing）

沒有為 8 個鄰居像素各自準備位址計算電路，而是共用一個 6-bit 加法器：

- gray_addr = center_addr + offset：
  - center_addr：目前的中心像素。
  - offset：根據 step 去。
- 在 8 x 8 的記憶體中，-9 代表左上方，+1 代表正右方。利用 look up table 的偏移量，電路只需一組加法器就能遍歷 3 x 3 的所有像素。負數用二補述去加，這樣就不用減法器

2. 共享比較器

之前我用了 8 個比較器，面積超大，後來就想到可以先讀中心的值，接下來，每讀一個新的值，就可以直接比較，只需一個比較器

- lbp <= { (gray_data >= gc), lbp[7:1] };

當處理第 1 個 g_0 時，結果放在 lbp[7]。

到了下一個週期處理 g_1 時，g_0 的結果會被往右推到 lbp[6]。

依此類推，處理完 8 個像素後，最先進入的 g_0 會剛好待在 lbp[0]（對應權重 2^0），最後進來的 g_7 則在 lbp[7]（對應 2^7）。

3. 邊界處理

為了節省暫存器，不使用 x 和 y 座標計數器（那需要 3+3=6 個 Flip-Flops），而是直接觀察位址規律：

- center_addr[2:0] == 3'd6：
  - 在 8 x 8 的陣列中，每一列的有效中心像素是座標 1 到 6。

    - 位址 6, 14, 22... 的共通點就是低 3 位元等於 6。

4. FSM

- IDLE：初始狀態。
- PROCESS：核心運算狀態。
    - step 0：讀取中心像素值並存入 gc。
    - step 1~8：依序讀取 g_0 到 g_7，並進行比較與位移。
- WRITE：運算完一個 LBP 值後，拉高 lbp_write 將結果存回 lbp_mem。此時也會判斷是否已處理到最後一個像素（位址 54）。
- DONE：全部完成，拉高 finish。

RTL sim 結果截圖(截到 CPU Time)

```
"verdi" : End of traversing.
----------------------------------------------------

START!!! Simulation Start .....

----------------------------------------------------

pixel           0 is CORRECT !! expected result is   0
pixel           1 is CORRECT !! expected result is   0
pixel           2 is CORRECT !! expected result is   0
pixel           3 is CORRECT !! expected result is   0
pixel           4 is CORRECT !! expected result is   0
pixel           5 is CORRECT !! expected result is   0
pixel           6 is CORRECT !! expected result is   0
pixel           7 is CORRECT !! expected result is   0
pixel           8 is CORRECT !! expected result is   0
pixel           9 is CORRECT !! expected result is 255
pixel          10 is CORRECT !! expected result is 192
pixel          11 is CORRECT !! expected result is 236
pixel          12 is CORRECT !! expected result is 254
pixel          13 is CORRECT !! expected result is 150
pixel          14 is CORRECT !! expected result is 214
pixel          15 is CORRECT !! expected result is   0
pixel          16 is CORRECT !! expected result is   0
pixel          17 is CORRECT !! expected result is 244
pixel          18 is CORRECT !! expected result is  16
pixel          19 is CORRECT !! expected result is   8
pixel          20 is CORRECT !! expected result is 252
pixel          21 is CORRECT !! expected result is 246
pixel          22 is CORRECT !! expected result is 150
pixel          23 is CORRECT !! expected result is   0
pixel          24 is CORRECT !! expected result is   0
pixel          25 is CORRECT !! expected result is 252
pixel          26 is CORRECT !! expected result is 246
pixel          27 is CORRECT !! expected result is  99
pixel          28 is CORRECT !! expected result is 105
pixel          29 is CORRECT !! expected result is 252
pixel          30 is CORRECT !! expected result is 150
pixel          31 is CORRECT !! expected result is   0
pixel          32 is CORRECT !! expected result is   0
pixel          33 is CORRECT !! expected result is 248
pixel          34 is CORRECT !! expected result is  96
pixel          35 is CORRECT !! expected result is  40
pixel          36 is CORRECT !! expected result is 105
pixel          37 is CORRECT !! expected result is 253
pixel          38 is CORRECT !! expected result is 150
pixel          39 is CORRECT !! expected result is   0
pixel          40 is CORRECT !! expected result is   0
pixel          41 is CORRECT !! expected result is   0
pixel          42 is CORRECT !! expected result is   8
pixel          43 is CORRECT !! expected result is  43
pixel          44 is CORRECT !! expected result is  41
pixel          45 is CORRECT !! expected result is  61
pixel          46 is CORRECT !! expected result is 150
pixel          47 is CORRECT !! expected result is   0
pixel          48 is CORRECT !! expected result is   0
pixel          49 is CORRECT !! expected result is   7
pixel          50 is CORRECT !! expected result is  11
pixel          51 is CORRECT !! expected result is  43
pixel          52 is CORRECT !! expected result is  43
pixel          53 is CORRECT !! expected result is  63
pixel          54 is CORRECT !! expected result is 151
pixel          55 is CORRECT !! expected result is   0
pixel          56 is CORRECT !! expected result is   0
pixel          57 is CORRECT !! expected result is   0
pixel          58 is CORRECT !! expected result is   0
pixel          59 is CORRECT !! expected result is   0
pixel          60 is CORRECT !! expected result is   0
pixel          61 is CORRECT !! expected result is   0
pixel          62 is CORRECT !! expected result is   0
----------------------------------------------------

Congratulations! All data have been generated successfully!

------------------------PASS------------------------

$finish called from file "testfixture.v", line 82.
$finish at simulation time              401500
           V C S   S i m u l a t i o n   R e p o r t
Time: 4015000 ps
CPU Time:      0.240 seconds;       Data structure size:   0.0Mb
```

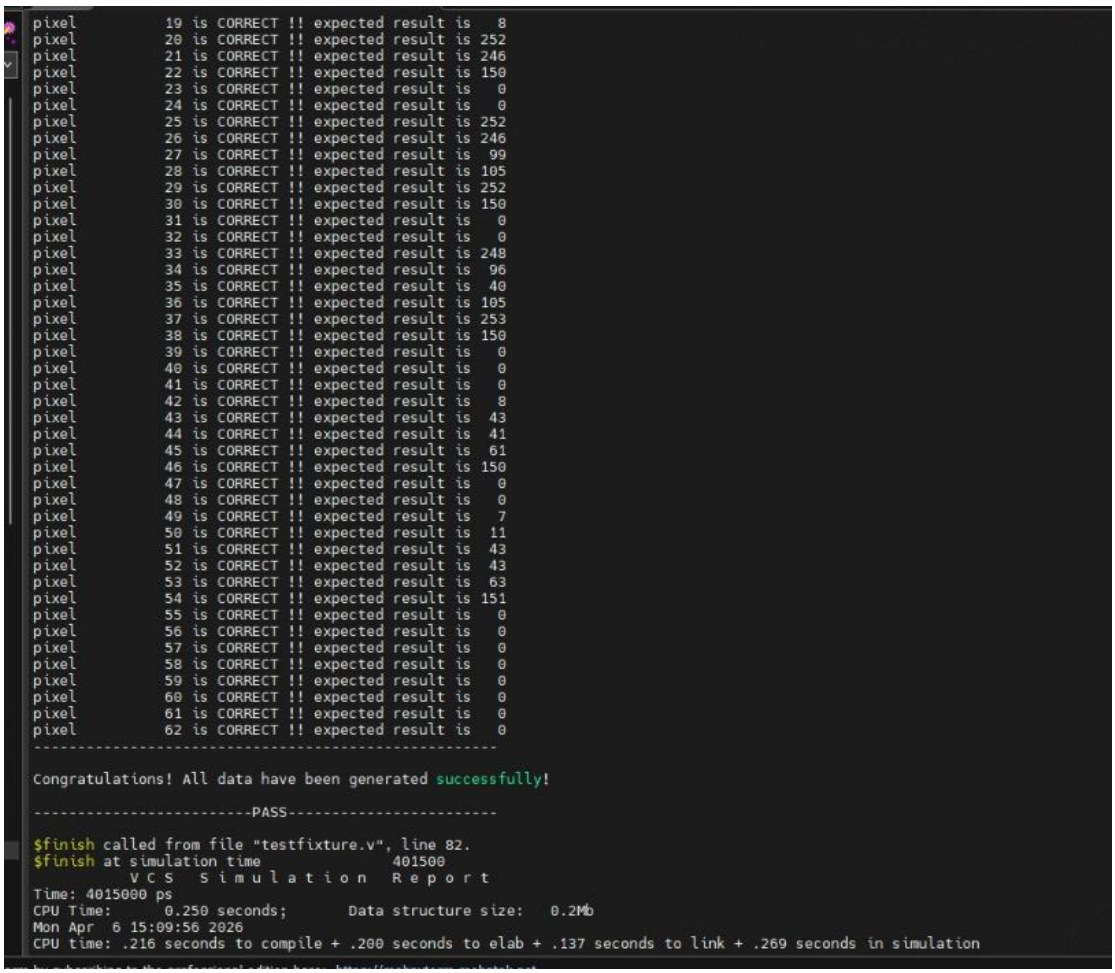
合成後 Report => timing_max 截圖

```
Point                                    Incr       Path
-----------------------------------------------------------
clock clk (rise edge)                    0.00       0.00
clock network delay (ideal)              3.00       3.00
input external delay                     5.00       8.00 f
gray_data[1] (in)                        0.01       8.01 f
U193/ZN (INVD1)                          0.03       8.04 r
U194/ZN (NR2D1)                          0.04       8.08 f
U195/ZN (NR2D1)                          0.04       8.12 r
U196/ZN (ND2D1)                          0.04       8.16 f
U198/ZN (ND2D1)                          0.04       8.20 r
U200/ZN (ND2D1)                          0.04       8.24 f
U203/ZN (ND4D1)                          0.04       8.28 r
U205/ZN (ND3D1)                          0.04       8.33 f
U207/ZN (ND2D1)                          0.04       8.37 r
U209/ZN (NR2D1)                          0.04       8.41 f
U210/Z (OR2D1)                           0.09       8.50 f
U211/ZN (ND2D1)                          0.04       8.55 r
U212/ZN (NR2D1)                          0.05       8.59 f
U213/ZN (NR2D1)                          0.05       8.64 r
U215/ZN (NR2D1)                          0.04       8.68 f
U217/ZN (ND2D1)                          0.03       8.71 r
U218/ZN (ND2D1)                          0.04       8.75 f
U219/Z (MUX2D1)                          0.12       8.87 f
lbp_reg[7]/D (DFCND4)                    0.00       8.87 f
data arrival time                                   8.87

clock clk (rise edge)                   10.00      10.00
clock network delay (ideal)              3.00      13.00
clock uncertainty                       -0.10      12.90
lbp_reg[7]/CP (DFCND4)                   0.00      12.90 r
library setup time                      -0.03      12.87
data required time                                 12.87
-----------------------------------------------------------
data required time                                 12.87
data arrival time                                  -8.87
-----------------------------------------------------------
slack (MET)                                         4.01
```

```
Library(s) Used:

    UDVS_u018mmbc198v (File: /cad/CBDK/CBDK_UMC018_ITRI_v0.8/CIC/SynopsysDC/db/UDVS_u018mmbc198v.db)

Number of ports:                          33
Number of nets:                          191
Number of cells:                         163
Number of combinational cells:           127
Number of sequential cells:               36
Number of macros/black boxes:              0
Number of buf/inv:                        16
Number of references:                     18

Combinational area:               2147.532846
Buf/Inv area:                      190.668799
Noncombinational area:            2674.380798
Macro/Black Box area:                0.000000
Net Interconnect area:      undefined  (Wire load has zero net area)

Total cell area:                  4821.913644
Total area:                 undefined

Hierarchical area distribution
------------------------------

                                  Global cell area          Local cell area
                                  ------------------  ----------------------------
Hierarchical cell                 Absolute   Percent  Combi-     Noncombi-  Black-
                                  Total      Total    national   national   boxes   Design
--------------------------------  ---------  -------  ---------  ---------  ------  ---------
LBP                               4821.9136    100.0  2147.5328  2674.3808  0.0000  LBP
--------------------------------  ---------  -------  ---------  ---------  ------  ---------
Total                                                 2147.5328  2674.3808  0.0000

1
```

GTL sim 結果截圖(截到 CPU Time)

```
Compiler version T-2022.06_Full64; Runtime version T-2022.06_Full64;  Apr  6 15:09 2026

Warning-[STASKW_RMIEAFK] Illegal entry
testfixture.v, 38
  More number of entries found at file ./golden1.dat line 64 while executing
  $readmem.
  Please ensure that the file has proper number of entries.
  The array name is exp_mem , size 63.

*Verdi* Loading libsscore_vcs202206.so
FSDB Dumper for VCS, Release Verdi_T-2022.06, Linux x86_64/64bit, 05/29/2022
(C) 1996 - 2022 by Synopsys, Inc.
*Verdi* FSDB WARNING: The FSDB file already exists. Overwriting the FSDB file may crash the programs that are using this file.
*Verdi* : Create FSDB file 'LBP.fsdb'
*Verdi* : Begin traversing the MDAs, layer (0).
*Verdi* : Enable +mda and +packedmda dumping.
*Verdi* : End of traversing the MDAs.
*Verdi* : Begin traversing the scopes, layer (0).
*Verdi* : End of traversing.
----------------------------------------------------

START!!! Simulation Start .....

----------------------------------------------------

pixel           0 is CORRECT !! expected result is   0
pixel           1 is CORRECT !! expected result is   0
pixel           2 is CORRECT !! expected result is   0
pixel           3 is CORRECT !! expected result is   0
pixel           4 is CORRECT !! expected result is   0
pixel           5 is CORRECT !! expected result is   0
pixel           6 is CORRECT !! expected result is   0
pixel           7 is CORRECT !! expected result is   0
pixel           8 is CORRECT !! expected result is   0
pixel           9 is CORRECT !! expected result is 255
pixel          10 is CORRECT !! expected result is 192
pixel          11 is CORRECT !! expected result is 236
pixel          12 is CORRECT !! expected result is 254
pixel          13 is CORRECT !! expected result is 150
pixel          14 is CORRECT !! expected result is 214
pixel          15 is CORRECT !! expected result is   0
pixel          16 is CORRECT !! expected result is   0
pixel          17 is CORRECT !! expected result is 244
pixel          18 is CORRECT !! expected result is  16
pixel          19 is CORRECT !! expected result is   8
pixel          20 is CORRECT !! expected result is 252
```



```
pixel          19 is CORRECT !! expected result is   8
pixel          20 is CORRECT !! expected result is 252
pixel          21 is CORRECT !! expected result is 246
pixel          22 is CORRECT !! expected result is 150
pixel          23 is CORRECT !! expected result is   0
pixel          24 is CORRECT !! expected result is   0
pixel          25 is CORRECT !! expected result is 252
pixel          26 is CORRECT !! expected result is 246
pixel          27 is CORRECT !! expected result is  99
pixel          28 is CORRECT !! expected result is 105
pixel          29 is CORRECT !! expected result is 252
pixel          30 is CORRECT !! expected result is 150
pixel          31 is CORRECT !! expected result is   0
pixel          32 is CORRECT !! expected result is   0
pixel          33 is CORRECT !! expected result is 248
pixel          34 is CORRECT !! expected result is  96
pixel          35 is CORRECT !! expected result is  40
pixel          36 is CORRECT !! expected result is 105
pixel          37 is CORRECT !! expected result is 253
pixel          38 is CORRECT !! expected result is 150
pixel          39 is CORRECT !! expected result is   0
pixel          40 is CORRECT !! expected result is   0
pixel          41 is CORRECT !! expected result is   0
pixel          42 is CORRECT !! expected result is   8
pixel          43 is CORRECT !! expected result is  43
pixel          44 is CORRECT !! expected result is  41
pixel          45 is CORRECT !! expected result is  61
pixel          46 is CORRECT !! expected result is 150
pixel          47 is CORRECT !! expected result is   0
pixel          48 is CORRECT !! expected result is   0
pixel          49 is CORRECT !! expected result is   7
pixel          50 is CORRECT !! expected result is  11
pixel          51 is CORRECT !! expected result is  43
pixel          52 is CORRECT !! expected result is  43
pixel          53 is CORRECT !! expected result is  63
pixel          54 is CORRECT !! expected result is 151
pixel          55 is CORRECT !! expected result is   0
pixel          56 is CORRECT !! expected result is   0
pixel          57 is CORRECT !! expected result is   0
pixel          58 is CORRECT !! expected result is   0
pixel          59 is CORRECT !! expected result is   0
pixel          60 is CORRECT !! expected result is   0
pixel          61 is CORRECT !! expected result is   0
pixel          62 is CORRECT !! expected result is   0
----------------------------------------------------

Congratulations! All data have been generated successfully!

-------------------------PASS------------------------

$finish called from file "testfixture.v", line 82.
$finish at simulation time              401500
           V C S   S i m u l a t i o n   R e p o r t
Time: 4015000 ps
CPU Time:      0.250 seconds;       Data structure size:   0.2Mb
Mon Apr  6 15:09:56 2026
CPU time: .216 seconds to compile + .200 seconds to elab + .137 seconds to link + .269 seconds in simulation
```

心得(以組為單位，一篇就好)

這次的作業難度提升了不少，想要寫好此程式必須先對此 LBP 的運作有深度了解過後，才再對每一種狀態進行分析與拆解，光是這幾個步驟就花了我許多的時間去了解。我的寫法步驟是先寫一部分的程式然後執行觀察波形是否與實際一樣，當成功之後才再繼續寫下一部分，而不是全部的程式打完才執行，這樣的作法對於除錯相較於容易一些。最後，整個 LBP 寫出來後，雖然說功能都正確無誤了，但是在合成後時間離目標還差了一大截，所以又回去修正一下我的 Verilog 的程式，把一些 for 迴圈跟 if else 換成 assign 、case 的方法寫，確實節省了一些時間，最後一直重複執行修改才達到目標的時間。經過 LBP 電路練習後，讓我學會有限狀態機的寫法與實際應用，除此之外，也學會如何讓電路更加簡化且能提升整個電路的運作速度。雖然此 LAB 花了我不少的時間，但是訓練下來真的對我受益良多。